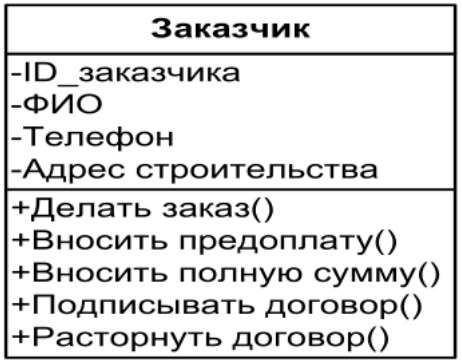
Лекция 5

UML (Unified Model Language)
Модели анализа, проектирования

ОСНОВНЫЕ ДИАГРАММЫ

диаграммы классов анализа, диаграммы взаимодействия (последовательности и коммуникации), диаграммы пакетов, модель проектирования, назначение и состав модели проектирования, диаграммы классов.

Диаграмма классов



- **Диаграмма классов** – основной способ описания структуры системы. Поскольку UML в первую очередь объектно—ориентированный язык, классы в нем являются основным, так называемым, «строительным материалом». Для построения диаграммы классов необходимо отобразить все выбранные классы. После этого необходимо составить список атрибутов и операций для выбранных классов.

Диаграммы классов

Наиболее распространенный тип диаграмм.

Описывает типы объектов системы и различного рода **статические** отношения, которые существуют между ними.

Помимо классов на диаграмме отображаются ***свойства классов***, ***операции классов*** и ***ограничения классов***.

Диаграмма классов

Диаграмма классов описывает типы объектов системы и различного рода статические отношения, которые существуют между ними.

На диаграмме прямоугольники представляют классы и разделены на три части:

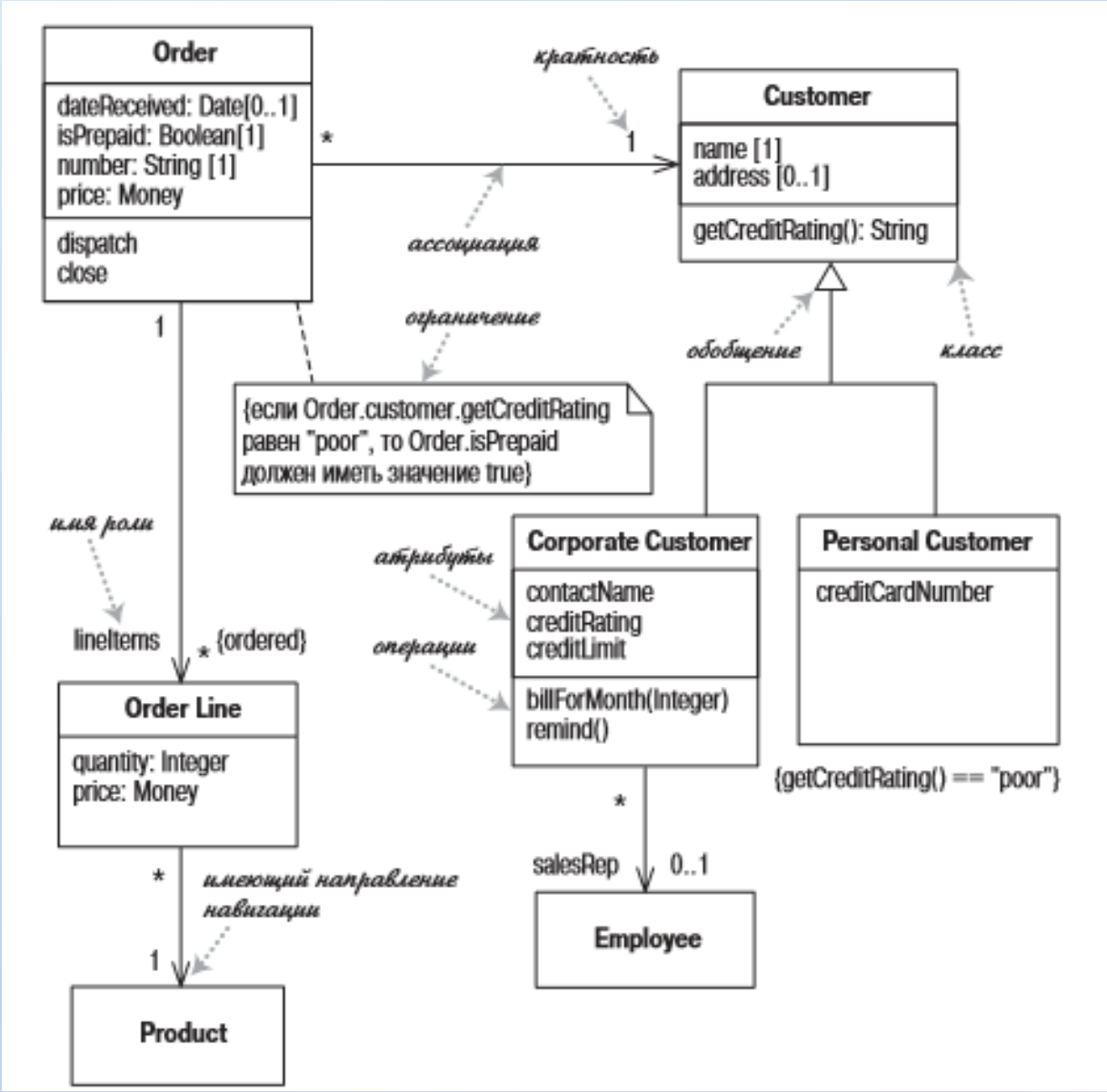
- имя класса(жирный шрифт),
- его атрибуты
- его операции.

На диаграммах классов отображаются также свойства классов, операции классов и ограничения, которые накладываются на связи между объектами.

В UML термин функциональность (feature) используется в качестве основного термина, описывающего и свойства, и операции класса.

Ниже изображена типичная модель класса, связанная с обработкой заказов клиентов.

Простая диаграмма классов



Выше на рисунке показаны два вида связей между классами: ассоциации и обобщения.

Свойства на диаграмме классов.

Свойства представляют структурную функциональность класса.

В первом приближении можно рассматривать свойства как поля класса.

Как мы увидим позднее, в действительности это не так просто, но вполне приемлемо для начала.

Свойства представляют единое понятие, воплощающееся в двух совершенно различных сущностях: в атрибутах и в ассоциациях.

Хотя на диаграмме они выглядят совершенно по-разному, в действительности это одно и то же.

Атрибуты

Атрибут описывает свойство в виде строки текста внутри прямоугольника класса.

• Метка видимость обозначает, относится ли атрибут к открытым (+) (public) или к закрытым (?) (private).

• Имя атрибута – способ ссылки класса на атрибут – приблизительно соответствует имени поля в языке программирования.

• Тип атрибута накладывает ограничение на вид объекта, который может быть размещен в атрибуте. Можно считать его аналогом типа поля в языке программирования.

• Значение по умолчанию представляет собой значение для вновь создаваемых объектов, если атрибут не определен в процессе создания.

• Элемент {строка свойств} позволяет указывать дополнительные свойства атрибута. В примере он равен readOnly, то есть клиенты не могут изменять атрибут. Если он пропущен, то обычно, атрибут можно модифицировать

• Кратность рассмотрена ниже.

Кратность

Кратность свойства обозначает количество объектов, которые могут заполнять данное свойство. Чаще всего встречаются следующие кратности: 0, 0…1, *.

Термины, имеющие отношение к кратности:

- Optional (необязательный) предполагает нулевую нижнюю границу.
- Mandatory (обязательный) подразумевает, что нижняя граница = или > 1.
- Singlevalued (однозначный) – для такого атрибута верхняя граница равна 1.
- Multivalued (многозначный) имеет в виду, что верхняя граница > 1; обычно *.

В большинстве случаев кратности определяются своими нижней и верхней границами. Нижняя граница может быть нулем или положительным числом, верхняя граница представляет собой положительное число или * (без ограничений). Если нижняя и верхняя границы совпадают, то можно указать одно число;

Ассоциации

Другая ипостась свойства – это ассоциация. Значительная часть информации, которую можно указать в атрибуте, появляется в ассоциации.

Ассоциация – это непрерывная линия между двумя классами, направленная от исходного класса к целевому классу. Имя свойства (вместе с кратностью) располагается на целевом конце ассоциации.

Целевой конец ассоциации указывает на класс, который является типом свойства. Большая часть информации в обоих представлениях одинакова, но некоторые элементы отличаются друг от друга. В частности, ассоциация может показывать кратность на обоих концах линии.

К другому распространенному типу ассоциаций относится двунаправленная ассоциация. Двунаправленная ассоциация – это пара свойств, связанных в противоположных направлениях.

(Пример Владелец ⟷ Имущество)

Операции

Операции (operations) представляют собой действия, реализуемые некоторым классом. Существует очевидное соответствие между операциями и методами класса. Обычно можно не показывать такие операции, которые просто манипулируют свойствами, поскольку они и такподразумеваются.

Полный синтаксис операций в языке UML выглядит следующим образом:

видимость имя (список параметров) : возвращаемый тип {строка свойств}

- Метка видимости обозначает, относится ли операция к открытым (+) (public) или к закрытым (?) (private);
- Имя – это строка.
- Список параметров – список параметров операции.
- Возвращаемый тип – тип возвращаемого значения, если таковое есть.
- Строка свойств – значения свойств, которые применяются к данной операции.

Операции

Операции (operations) представляют собой действия, реализуемые некоторым классом. Существует очевидное соответствие между операциями и методами класса. Обычно можно не показывать такие операции, которые просто манипулируют свойствами, поскольку они и такподразумеваются.

Полный синтаксис операций в языке UML выглядит следующим образом:

видимость имя (список параметров) : возвращаемый тип {строка свойств}

- Метка видимости обозначает, относится ли операция к открытым (+) (public) или к закрытым (?) (private);
- Имя – это строка.
- Список параметров – список параметров операции.
- Возвращаемый тип – тип возвращаемого значения, если таковое есть.
- Строка свойств – значения свойств, которые применяются к данной операции.

Зависимость

Считается, что между двумя элементами существует зависимость (dependency),

если изменения в определении одного элемента (сервера)

могут вызвать изменения в другом элементе (клиенте).

В случае классов зависимости появляются по разным причинам:

- один класс посылает сообщение другому классу;

- один класс владеет другим классом как частью своих данных;

- один класс использует другой класс в качестве параметра операции.

UML позволяет изобразить зависимости между элементами всех типов.

Зависимости можно использовать всякий раз, когда надо показать,

как изменения в одном элементе могут повлиять на другие элементы.

Правила ограничений

При построении диаграмм классов большая часть времени уходит на представление различных ограничений. Например на рисунке показано, что Заказ (Order) может быть сделан только одним единственным Клиентом (Customer).

Из этой диаграммы классов также следует, что каждая Line Item (Позиция заказа) рассматривается отдельно. Также диаграмма показывает, что Корпоративные клиенты располагают кредитами, а Индивидуальные клиенты – нет.

С помощью базовых конструкций ассоциации, атрибута и обобщения можно сделать многое, специфицируя наиболее важные ограничения, но этими средствами невозможно записать каждое ограничение.

Эти ограничения еще нужно каким-то образом отобразить, и диаграмма классов является вполне подходящим местом для этого.

Правила ограничений

Язык UML разрешает использовать для описания ограничений все что угодно.

При этом необходимо лишь придерживаться правила: ограничения следует помещать в фигурные скобки ({}).

Можно употреблять разговорный язык, язык программирования или формальный объектный язык ограничений UML (Object Constraint Language, OCL)

Формальное написание позволяет избежать риска неоднозначного толкования конструкций разговорного языка.

Можно предварять ограничение именем с двоеточием

Пример готовой диаграммы классов

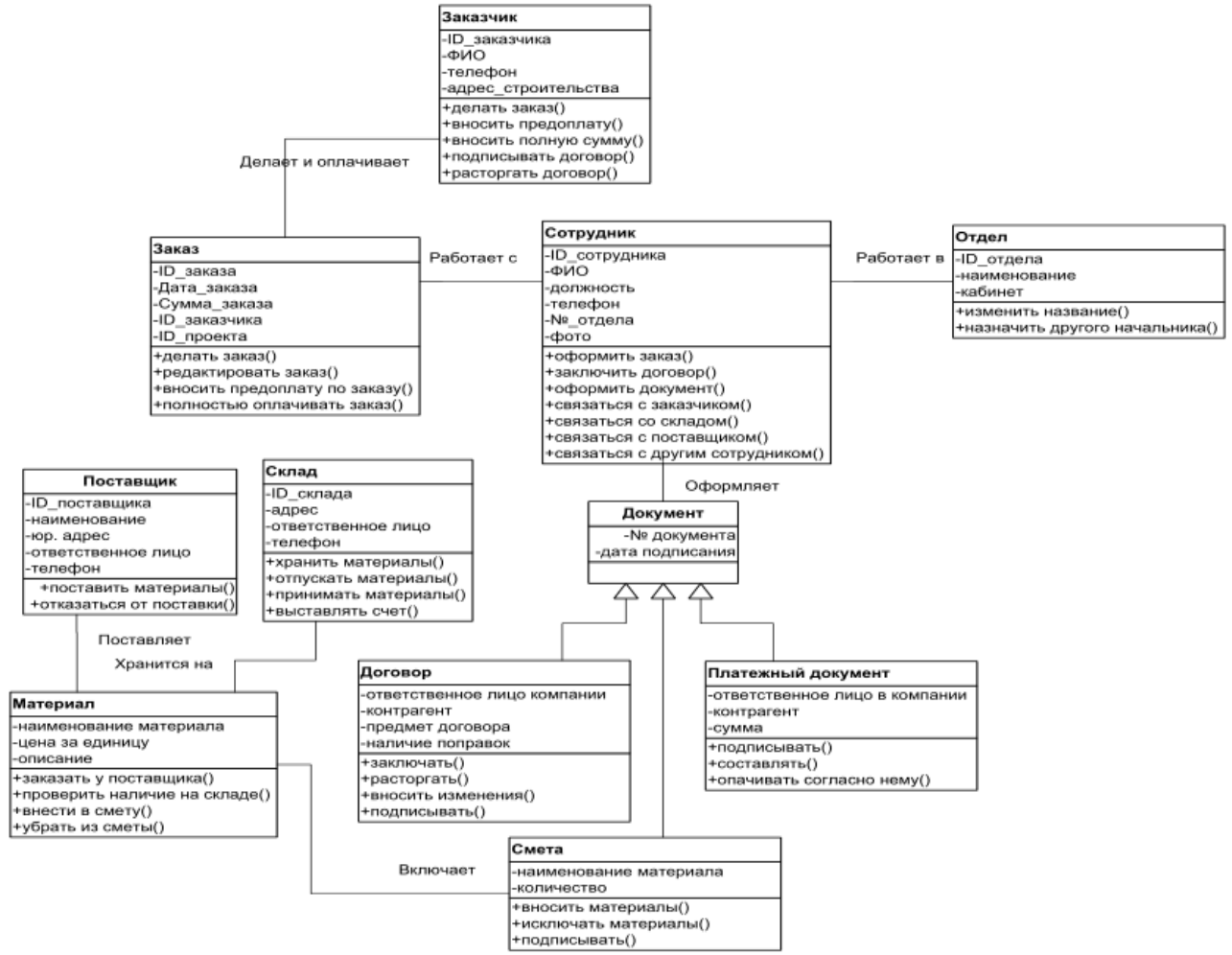


Диаграмма анализа классов

Фундаментальными понятиями объектно-ориентированного подхода являются понятия *объекта* и *класса* которые представляются абстракциями реальной или воображаемой сущности (набора сущностей).

Класс анализа – еще более абстрактная сущность, чем просто класс, представляет собой набор из одного или более классов.

Таким образом, ***класс анализа*** – это укрупненная абстракция, которая на концептуальном уровне (без точного определения атрибутов и операций) описывает некоторый фрагмент системы.

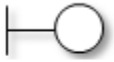
Существует три вида классов анализа:

- граничный;
- управляющий;
- сущности.

Диаграмма анализа классов

Диаграмма классов анализа по существу является прообразом классической диаграммы классов

Элементами, отображаемыми на диаграмме, являются классы и отношения между ними.

Варианты отображения	Граничный класс	Управляющий класс	Класс сущности
Графический стереотип	 Диалоговое окно «Нормативы»	 Расчет Vдоп	 План
Стандартное обозначение со строкой-стереотипом	«boundary» Диалоговое окно «Нормативы»	«control» Расчет Vдоп	«entity» План

При этом для отображения классов можно воспользоваться стандартным обозначением класса (прямоугольник) с указанием внутри него соответствующего стереотипа или значком-стереотипом.

Диаграмма анализа классов

Рассмотренное разбиение классов анализа на три группы согласуется с популярным шаблоном проектирования **«Модель-Представление-Контроллер»** (англ. MVC, Model-View-Controller).

При этом под классом сущности - понимается модель, граничным классом - представление и управляющим классом - контроллер.

Связи между классами анализа отображаются с использованием ***отношений*** пяти видов:

- ассоциаций;
- агрегаций;
- композиций;
- обобщения;
- зависимостей.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

1. При выделении классов анализа следует учитывать тот факт, что они являются обобщенными (укрупненными) сущностями, которые в дальнейшем подлежат уточнению и возможному разбиению на несколько более мелких классов. Можно провести аналогию между ними и накопителями данных на DFD, которые, как правило, являются крупными концептуальными объектами (совокупностями тесно взаимосвязанных таблиц), подлежащими уточнению на стадии информационного проектирования с помощью одной из методологий ERD.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

2. Для выделения классов сущностей необходимо определить все реальные либо воображаемые объекты, имеющие существенное значение для рассматриваемой предметной области, информация о которых подлежит хранению. При этом из спецификаций вариантов использования следует выделить все объекты, которые могут существовать независимо от других. Например, объект «билет» является независимой сущностью, потому что любой билет существует независимо от того, знаем мы его номер, стоимость или нет. Т. е. при выделении классов-сущностей действуют те же правила, что при построении концептуальной модели БД.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

3. Для каждого актера следует предусмотреть, как минимум, один граничный класс в целях организации интерфейса между ним и системой. Аналогично для каждого класса сущности, как правило, должен быть граничный класс – ведь по каждому объекту класса сущности должна быть предусмотрена возможность просмотра, ввода и/или корректировки информации через определенную форму ввода/вывода или чтения/записи через определенный интерфейс.

4. Для управления, обеспечения взаимодействия и координации работы объектов, реализующих одну из функций системы (обычно, вариант использования), необходимо предусмотреть, как минимум, один управляющий класс. Как правило, взаимодействие между граничным классом и классом сущности происходит через управляющий класс.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

5. В целях облегчения восприятия специфики связей между классами рекомендуется использовать отношения агрегации, композиции и обобщения.

6. При разработке диаграммы основное внимание должно быть уделено определению и детализации классов сущностей, управляющих и граничных классов, обеспечивающих взаимодействие с внешними системами.

Граничные классы, обеспечивающие взаимодействие с пользователями, не требуют излишней детализации до уровня отдельного поля ввода или ниспадающего списка, так как современные среды программирования обладают богатыми возможностями по быстрому созданию пользовательского интерфейса.

Диаграмма анализа классов. Пример

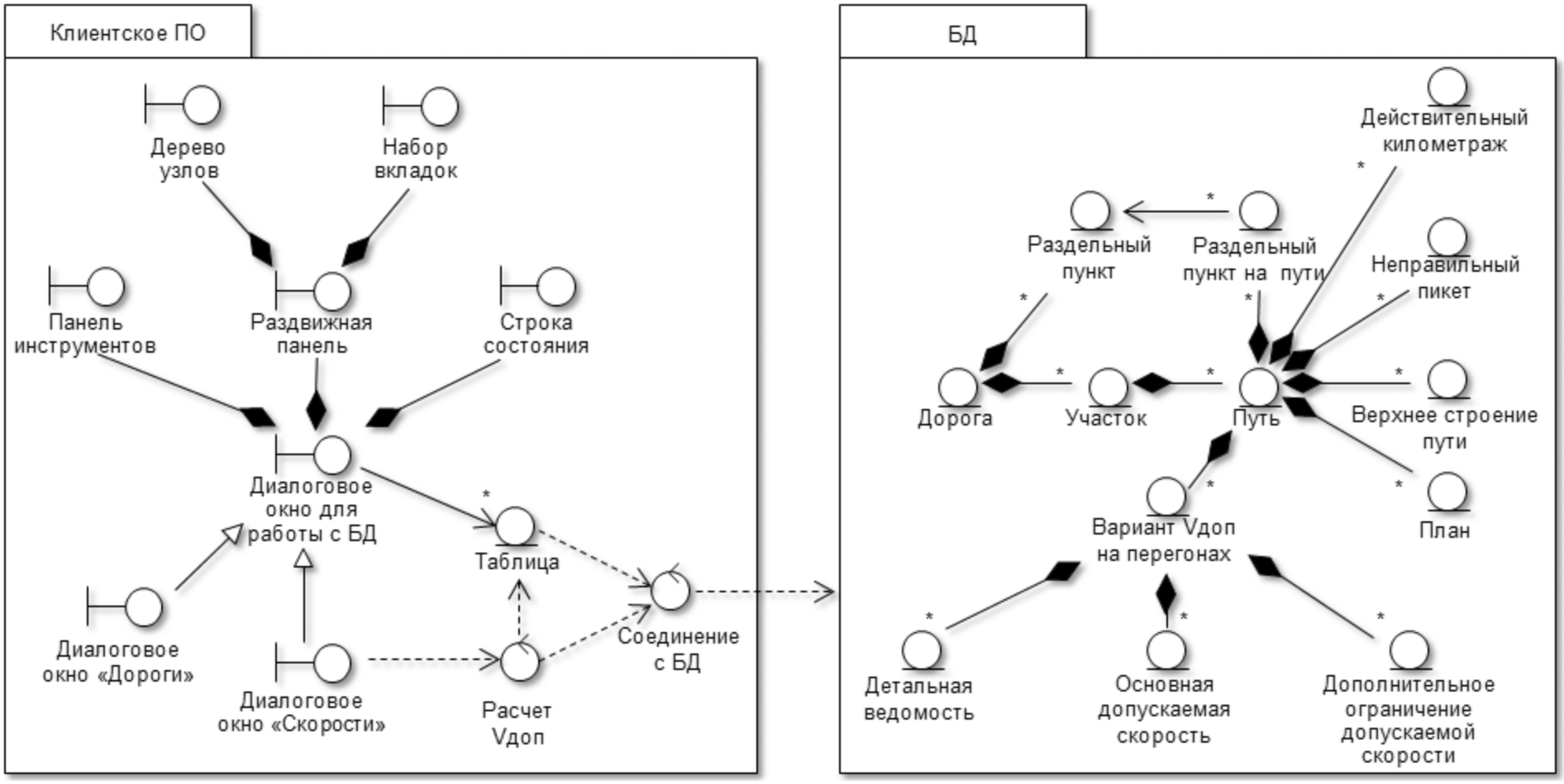


Диаграмма анализа классов

В правой части (начиная с класса «Дорога») расположены классы, описывающие структуру БД на сервере. Хотя исходный программный код не будет содержать этих классов, их детальная разработка не менее важна, чем разработка остальных классов. В данном случае правая часть диаграммы представляет концептуальную модель БД, которую можно описать с помощью одной из методологий ERD.

Во время работы программы информация из БД считывается в объекты класса «Таблица» (кэшируется), обрабатывается объектами других классов и при необходимости обратно записывается в БД. Каждый такой объект, помимо данных конкретной таблицы БД, содержит описание свойств таблицы в целом, полей (имя поля, его тип, значение по умолчанию и т. д.), описание структуры заголовка таблицы, отображаемого на экране, и другую необходимую информацию.

За каждым диалоговым окном, предназначенным для работы с БД, закрепляется определенный набор объектов класса «Таблица».

Диаграмма анализа классов

Аналогично классу «Диалоговое окно "Дороги"» в левую часть диаграммы должны быть нанесены классы «Диалоговое окно "Справочники"» и «Диалоговое окно "Нормативы"», а аналогично классу «Диалоговое окно "Скорости"» – класс «Диалоговое окно "Возвышения"», связанный с управляющим классом «Расчет возвышений».

Концептуальная модель БД должна быть дополнена таблицами, описывающими нормативно-справочную информацию и остальные данные дорожного уровня, необходимые для работы системы.

Диаграммы последовательности

Диаграммы взаимодействия (interaction diagrams) описывают взаимодействие групп объектов в различных условиях их поведения. UML определяет диаграммы взаимодействия нескольких типов, из которых наиболее употребительными являются диаграммы последовательности.

Обычно диаграмма последовательности описывает один сценарий.

На диаграмме показаны экземпляры объектов и сообщения, которыми обмениваются объекты в рамках одного прецедента (use case).

Предположим, что у нас есть заказ, и мы собираемся вызвать команду для определения его стоимости. При этом объекту заказа (Order) необходимо просмотреть все позиции заказа (Line Items) и определить их цены, основанные на правилах построения цены продукции в строке заказа (Order Line).

Проделав это для всех позиций заказа, объект заказа должен вычислить общую скидку, которая определяется индивидуально для каждого клиента.

Диаграммы последовательности

Диаграммы взаимодействия (interaction diagrams) описывают взаимодействие групп объектов в различных условиях их поведения. UML определяет диаграммы взаимодействия нескольких типов, из которых наиболее употребительными являются диаграммы последовательности.

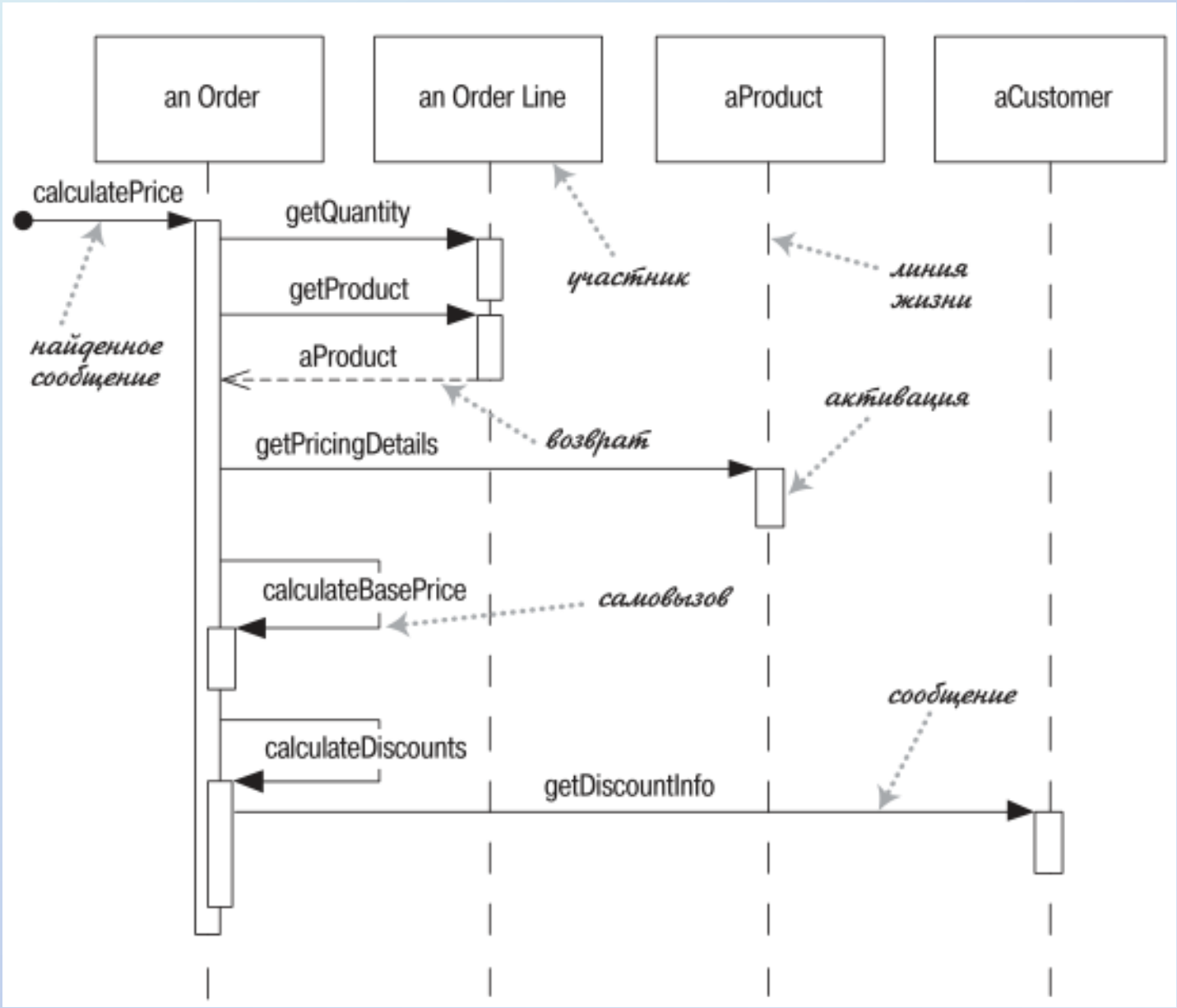
Обычно диаграмма последовательности описывает один сценарий.

На диаграмме показаны экземпляры объектов и сообщения, которыми обмениваются объекты в рамках одного прецедента (use case).

Предположим, что у нас есть заказ, и мы собираемся вызвать команду для определения его стоимости. При этом объекту заказа (Order) необходимо просмотреть все позиции заказа (Line Items) и определить их цены, основанные на правилах построения цены продукции в строке заказа (Order Line).

Проделав это для всех позиций заказа, объект заказа должен вычислить общую скидку, которая определяется индивидуально для каждого клиента.

Диаграммы последовательности. Пример



Диаграммы последовательности

Диаграммы последовательности показывают взаимодействие, представляя каждого участника вместе с его линией жизни (lifeline), которая идет вертикально вниз и упорядочивает сообщения на странице;

Сообщения также следует читать сверху вниз, что понятно на диаграмме

Видно, что экземпляр заказа посылает строке заказа сообщения getQuantity и getProduct. Можно увидеть также, как заказ применяет метод к самому себе и как этот метод посылает сообщение getDiscountInfo экземпляру клиента.

Однако диаграмма не все показывает так хорошо. Последовательность сообщений getQuantity, getProduct, getPricingDetails и calculateBasePrice должна быть реализована для каждой строки заказа, тогда как метод calculateDiscounts вызывается лишь однажды.

Диаграммы последовательности

В большинстве случаев можно считать участников диаграммы взаимодействия объектами, как это и было в действительности в UML 1.

Однако в UML 2 их роль значительно сложнее, и полное ее объяснение выходит за рамки этой книги. Поэтому я употребляю термин участники (participants), который формально не входит в спецификацию UML.

В UML версии 1 участники были объектами, и поэтому их имена подчеркивались, но в UML 2 их надо показывать без подчеркивания.

На приведенной диаграмме именуют участников, используя стиль anOrder.

В большинстве случаев это вполне приемлемо.

Вот более полный синтаксис: имя : Класс, где и имя, и класс не обязательны, но если класс используется, то двоеточие должно присутствовать.

У первого сообщения нет участника, пославшего его, так как оно приходит из неизвестного источника. Оно называется найденным сообщением (found message).

Диаграммы последовательности

Каждая линия жизни имеет полосу активности, которая показывает интервал активности участника при взаимодействии. Она соответствует времени нахождения в стеке одного из методов участника. В языке UML полосы активности не обязательны Именование бывает часто полезным для установления связей между участниками на диаграмме. Как видно на диаграмме, вызов метода getProduct возвращает aProduct, имеющего то же самое имя и, следовательно, означающего того же самого участника, aProduct, которому по сылается вызов getPricingDetails. **Обратной стрелкой обозначен только этот вызов с целью показать соответствие.**

Многие разработчики используют возвраты для всех вызовов, но лучше применять их, только когда это дает дополнительную информацию; иначе они просто вносят неразбериху.

Возможно, что даже в данном случае можно было опустить возврат.

Диаграммы последовательности

Основная задача остается той же самой, но способ взаимодействия участников для ее решения совершенно другой.

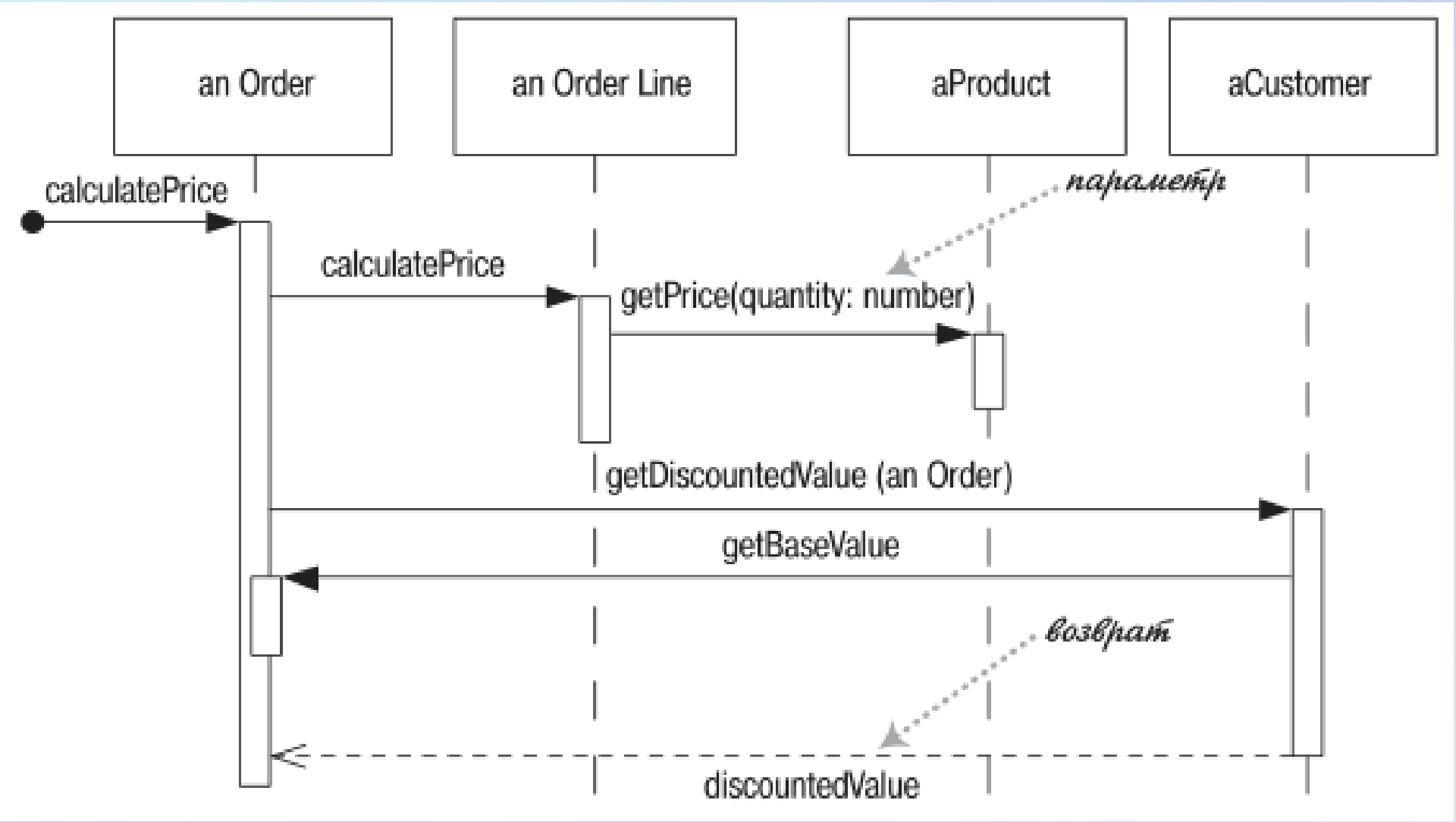
Заказ спрашивает каждую строку заказа о его собственной цене (Price).

Сама строка заказа передает вычисление дальше – объекту продукта (Product); обратите внимание, как мы показываем передачу параметра.

Подобным же образом для вычисления скидки объект заказа вызывает метод для клиента (Customer).

Поскольку для выполнения этой задачи клиенту требуется информация от объекта заказа, то он делает повторный вызов в отношении заказа для получения этих данных.

Диаграммы последовательности



Диаграммы последовательности

На этих двух диаграммах надо обратить внимание, на сколько ясно диаграмма последовательности показывает различия во взаимодействии участников. В этом проявляется мощь диаграмм взаимодействий. Они не очень хорошо представляют детали алгоритмов, такие как циклы или условное поведение, но делают абсолютно прозрачными вызовы между участниками и дают действительно ясную картину того, какую обработку выполняют конкретные участники.

Во-вторых, разница в стиле между двумя взаимодействиями заключается также в том, что на первом рисунке представлено централизованное управление (centralized control), когда один из участников в значительной степени выполняет всю обработку, а другие предоставляют данные. На втором рисунке изображено распределенное управление (distributed control), при котором обработка распределяется между многими участниками, каждый их которых выполняет небольшую часть алгоритма.

Диаграммы последовательности

Большинство разработчиков, особенно новички в объектно-ориентированном программировании, чаще всего применяют централизованное управление.

Во многих случаях это проще, так как вся обработка сосредоточена в одном месте; напротив, в случае распределенного управления при попытке понять программу создается ощущение погони за объектами.

Кроме того, распределенное управление позволяет создать больше возможностей для применения полиморфизма, чем в случае применения условной логики.

Если алгоритмы определения цены отличаются для различных типов продуктов, то механизм распределенного управления позволяет нам использовать подклассы класса продукта (Product) для обработки этих вариантов.

Диаграммы последовательности

Объектно-ориентированный стиль предназначен для работы с большим количеством небольших объектов, обладающих множеством небольших методов, что дает широкие возможности для переопределения и изменения.

Этот стиль сбивает с толку людей, применяющих длинные процедуры; действительно это изменение является сердцем смены парадигмы (paradigm shift) при объектной ориентации.

Научить этому трудно, возможно, что единственный способ действительно понять это заключается в использовании распределенного управления при работе в объектно-ориентированном окружении.

Диаграммы последовательности

В диаграммах последовательности для создания и удаления участников применяются некоторые дополнительные обозначения.

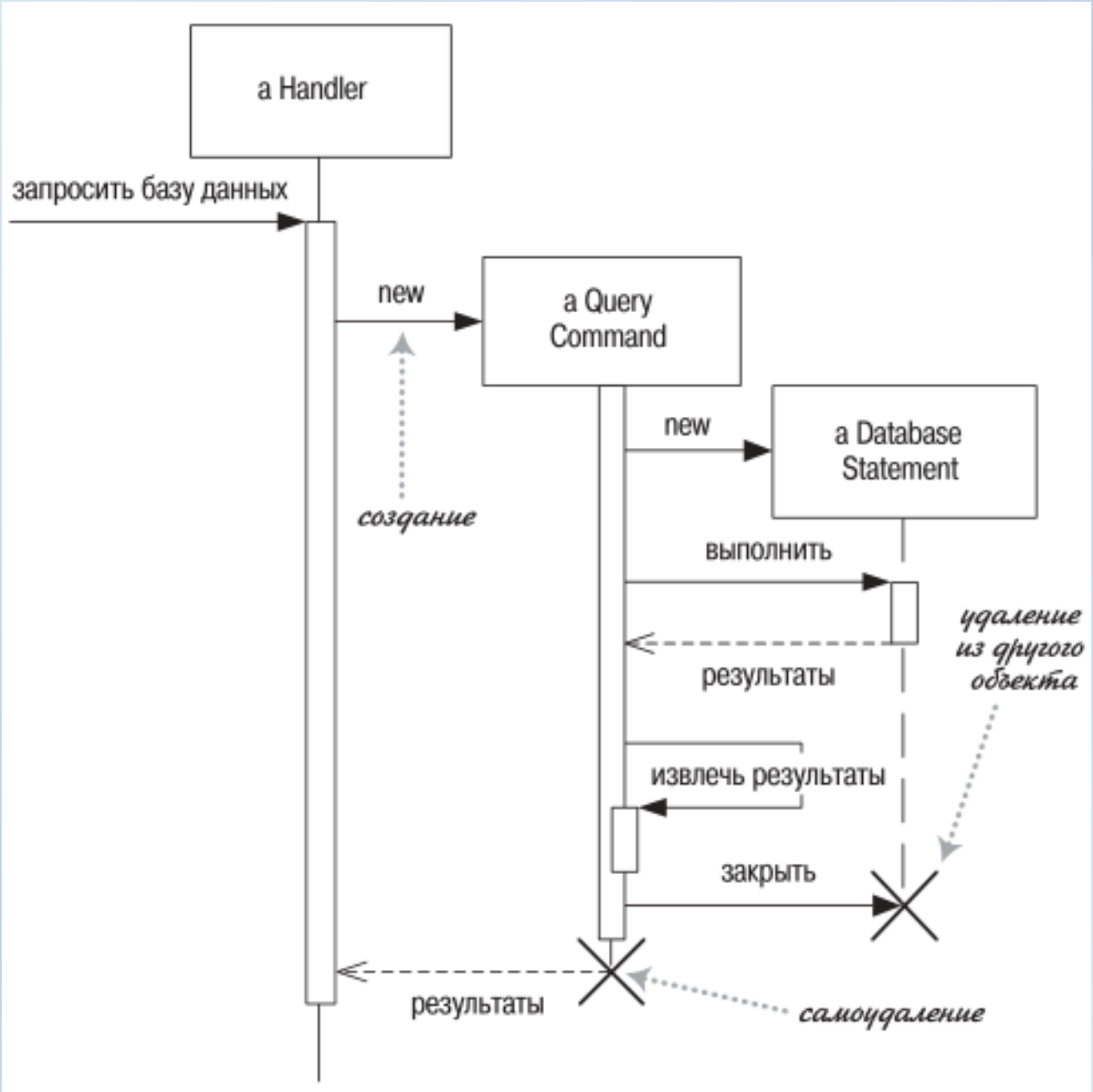
В случае создания участника надо нарисовать стрелку сообщения, направленную к прямоугольнику участника.

Удаление участника обозначается большим крестом (X). Стрелка сообщения, идущая в X, означает, что один участник явным образом удаляет другого; X в конце линии жизни показывает, что участник удаляет сам себя.

Если в системе работает сборщик мусора, то объекты не удаляются вручную, тем не менее следует при помощи X показать, что объект больше не нужен и готов к удалению.

Так следует поступать и в случае операций закрытия, показывая, что объект больше не используется.

Диаграммы последовательности



Диаграммы последовательности

Общая проблема диаграмм последовательности заключается в том,

как отображать циклы и условные конструкции.

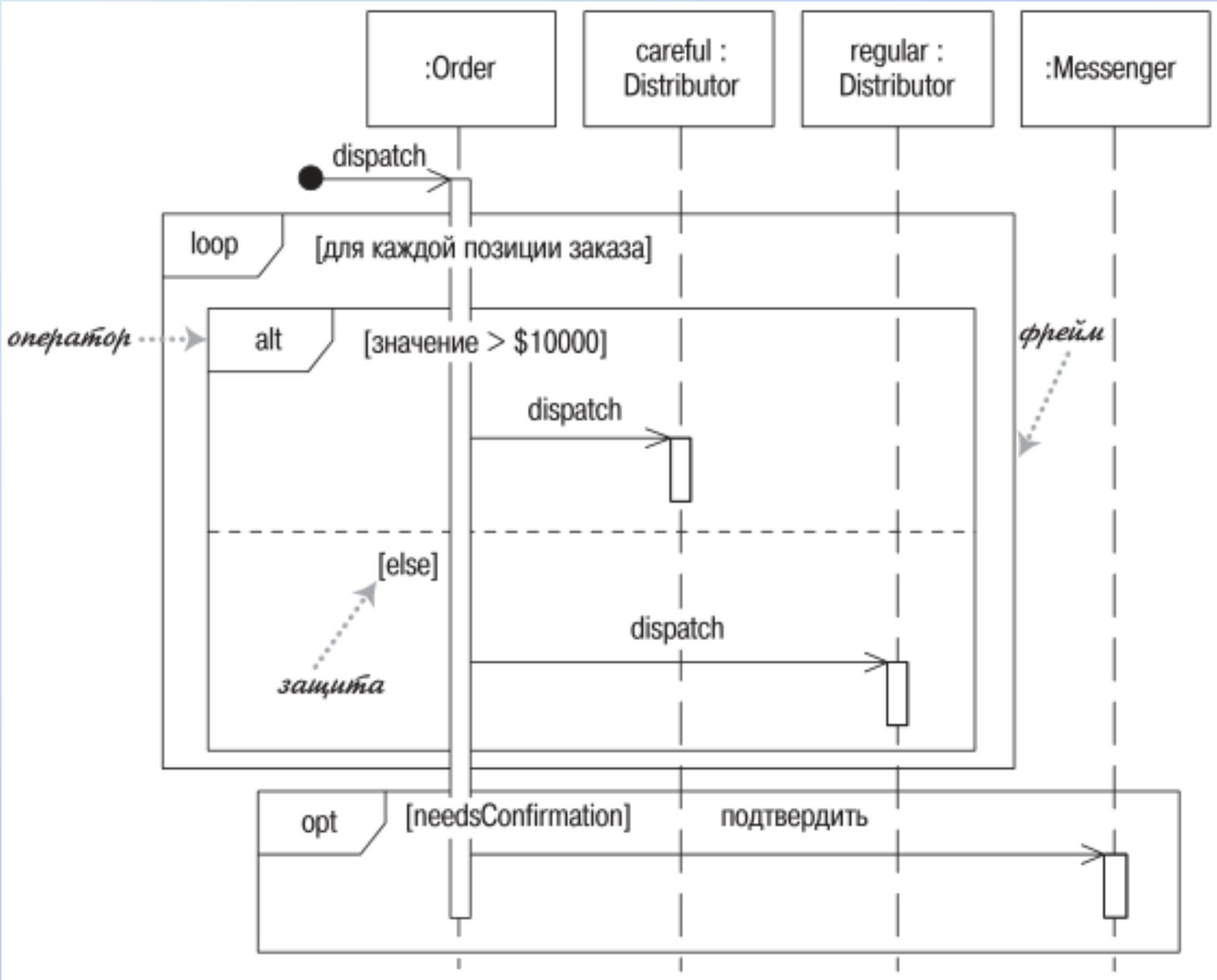
Прежде всего надо усвоить, что диаграммы последовательности для этого не предназначены.

Подобные управляющие структуры лучше показывать с помощью диаграммы деятельности или собственно кода.

Диаграммы последовательности применяются для визуализации процесса взаимодействия объектов, а не как средство моделирования алгоритма управления.

Как было сказано, существуют дополнительные обозначения. И для циклов, и для условий используются фреймы взаимодействий (interaction frames), представляющие собой средство разметки диаграммы взаимодействия.

Диаграммы последовательности



Диаграммы последовательности

В основном фреймы состоят из некоторой области диаграммы последовательности, разделенной на несколько фрагментов.

Каждый фрейм имеет оператор, а каждый фрагмент может иметь защиту.

Ниже в таблице перечислены общепринятые операторы для фреймов взаимодействия.

Для отображения цикла применяется оператор loop с единственным фрагментом, а тело итерации помещается в защиту.

Для условной логики можно использовать оператор alt и помещать условие в каждый фрагмент. Будет выполнен только тот фрагмент, защита которого имеет истинное значение.

Для единственной области существует оператор opt.

Диаграммы последовательности

Оператор	Значение
`alt`	Несколько альтернативных фрагментов (alternative); выполняется только тот фрагмент, условие которого истинно
`opt`	Необязательный (optional) фрагмент; выполняется, только если условие истинно. Эквивалентно `alt` с одной веткой
`par`	Параллельный (parallel); все фрагменты выполняются параллельно
`loop`	Цикл (loop); фрагмент может выполняться несколько раз, а защита обозначает тело итерации
`region`	Критическая область (critical region); фрагмент может иметь только один поток, выполняющийся за один прием
`neg`	Отрицательный (negative) фрагмент; обозначает неверное взаимодействие
`ref`	Ссылка (reference); ссылается на взаимодействие, определенное на другой диаграмме. Фрейм рисуется, чтобы охватить линии жизни, вовлеченные во взаимодействие. Можно определять параметры и возвращать значение
`sd`	Диаграмма последовательности (sequence diagram); используется для очерчивания всей диаграммы последовательности, если это необходимо

Диаграммы последовательности

Фреймы взаимодействия – новинка UML 2. В диаграммах, разработанных до создания UML 2, применяется другой подход; кроме того, некоторые разработчики не любят фреймы и предпочитают прежние соглашения.

Ниже на рисунке показаны некоторые из этих неофициальных приемов.

В UML 1 использовались маркеры итераций и защиты. В качестве маркера итерации (iteration marker) выступал символ *, добавленный к имени сообщения. Для обозначения тела итерации можно добавить текст в квадратных скобках. Защита (guard) – это условное выражение, размещенное в квадратных скобках и означающее, что сообщение посылается, только когда защита принимает истинное значение.

Эти обозначения исключены из UML 2, но они все еще встречаются в диаграммах взаимодействия.

Диаграммы последовательности

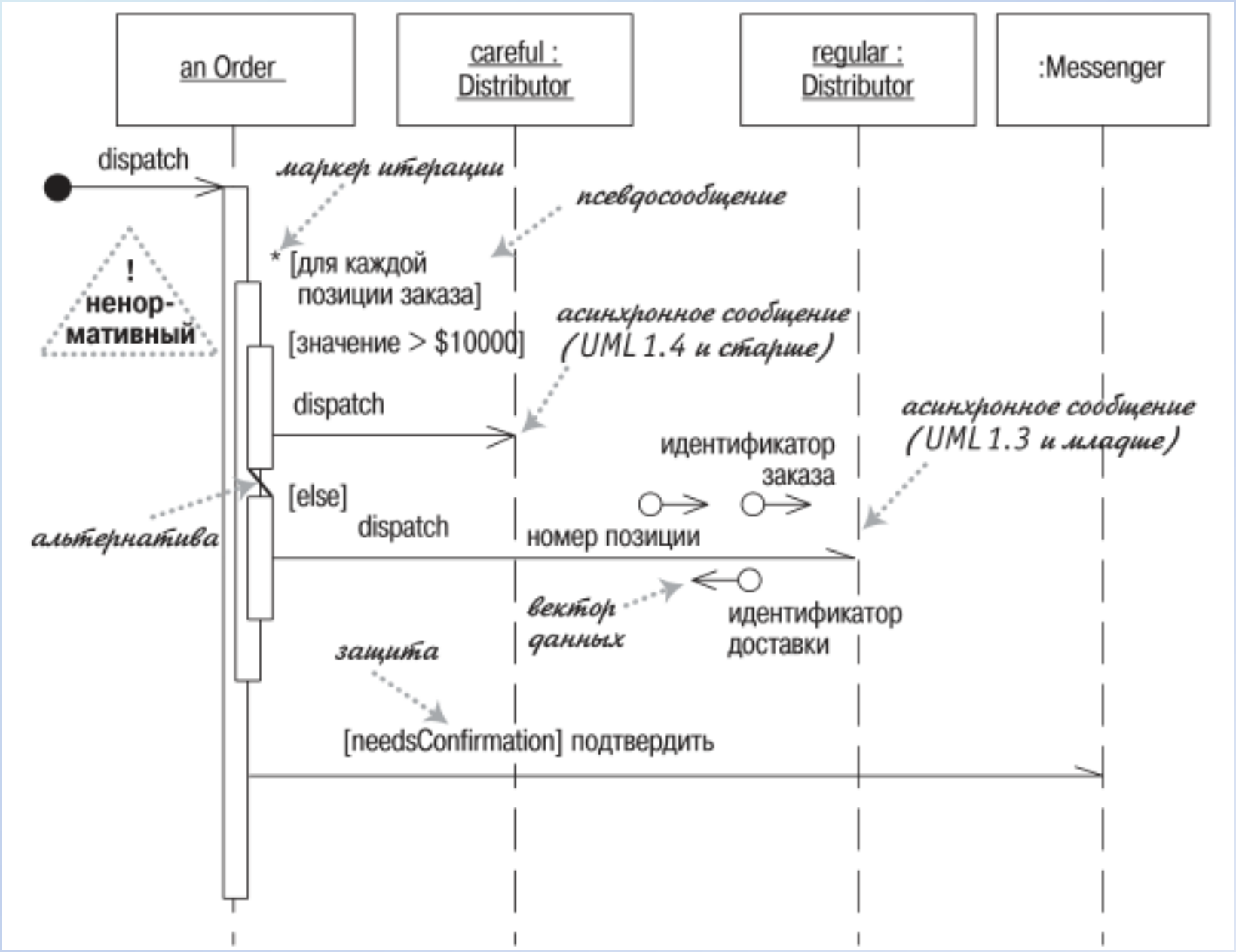
Несмотря на то что маркеры итерации и защиты могут оказаться полезными, они имеют один недостаток.

С помощью защиты нельзя показать, что несколько защит взаимно исключают друг друга.

Оба обозначения работают только в случае отправки одного сообщения и не работают, когда при одной активации посылается несколько сообщений в рамках того же самого цикла или условного блока.

Решением последней проблемы может служить ставшее популярным неофициальное соглашение, заключающееся в применении псевдо-сообщения (pseudomessage) в виде условия цикла или защиты на одном из вариантов обозначения самовызова.

Диаграммы последовательности



Диаграммы последовательности

Стандарт UML не предоставляет графических средств для обозначения передаваемых данных; вместо этого они показываются с помощью параметров в имени сообщения и на стрелках возврата.

Векторы данных повсеместно использовались во многих областях для обозначения перемещения данных, и многие разработчики все еще с удовольствием применяют их в UML.

В конечном счете, хотя различные системы могут включать в диаграммы последовательности обозначения для условной логики, я не думаю, что они работают сколько-нибудь лучше программного кода или, по крайней мере, псевдокода.

Пример готовой диаграммы классов

